CADERNO DE ESTUDO • DESENVOLVIMENTO WEB

HTML, JavaScript e banco de dados

Do campo preenchido ao SQL.
Da consulta à informação na tela.



SELECT → objetos JavaScript → elementos atualizados no DOM

Projeto de referência: Universidade Comunitária

Apostila ampliada com recortes numerados dos arquivos originais, explicações passo a passo, exemplos de evolução e 50 exercícios de SQL.

O que você aprenderá
Localizar elementos HTML, capturar eventos, ler e validar valores, executar comandos parametrizados, consultar o SQLite e inserir o retorno no lugar correto da página.

Material em português • nível introdutório a intermediário
Base documental: index.html, app.js, db.js, DDL.sql, DML.sql e DQL.sql enviados pelo usuário.

Como usar esta apostila

Leia os capítulos em sequência na primeira utilização. Durante a prática, use os números de linha para localizar o mesmo trecho nos arquivos enviados. Os recortes mostram o código existente; os exemplos marcados como adaptação ou extensão mostram possibilidades de melhoria.

Identificação	Como interpretar
Recorte original	Trecho extraído diretamente de um arquivo enviado. A numeração é do arquivo de origem.
Exemplo didático • adaptação	Código simplificado ou melhorado para explicar um conceito. Não representa necessariamente o comportamento atual.
Extensão proposta	Funcionalidade adicional que exige integração ao projeto. Não deve ser colada junto a um listener equivalente já existente.
Atenção ao projeto	Limite ou problema observado na implementação original.

Objetivos de aprendizagem

• Relacionar cada ID do HTML ao trecho JavaScript que o utiliza.

• Explicar a diferença entre elemento DOM, valor do campo, objeto JavaScript e linha de tabela SQL.

• Percorrer o fluxo completo de INSERT, SELECT, UPDATE e DELETE.

• Interpretar chaves, relacionamentos, validações e efeitos de exclusão.

• Criar consultas de complexidade crescente com base nos arquivos SQL.

Sugestão de percurso para as aulas

Etapa	Atividade
1	Preparação, DOM e eventos.
2	Captura de pessoa e disciplina; INSERT parametrizado.
3	SELECT e apresentação do retorno; preenchimento para edição.
4	Relacionamentos, UPDATE, DELETE e integridade.
5	Laboratório de integração e bateria final de SQL.

Escopo
O banco deste exemplo executa no navegador. A comparação com API aparece para situar a arquitetura, sem atribuir ao projeto um servidor que ele não possui.

Sumário

Como usar esta apostila 2
1. Arquitetura: quem faz o quê? 4
2. Preparação e ordem de carregamento 5
3. DOM: encontrando o campo certo 6
4. Inicialização, eventos e envio 7
5. Captura de pessoa: HTML → objeto 8
6. Tipos, conversões e validações 9
7. INSERT: objeto → parâmetros → banco 10
8. Dentro de DB: memória e execução 11
9. SELECT: linhas → objetos JavaScript 13
10. Onde o retorno entra no HTML? 14
11. Montando uma tabela dinâmica 15
12. Caso completo: disciplina 16
13. Estudantes, matrículas e orientação 18
14. Do banco para campos de edição 20
15. UPDATE, DELETE e sincronização 21
16. Arquivos SQL e console da página 22
17. Modelo relacional e regras do DDL 23
18. SQL de leitura: do básico aos relatórios 24
19. Depuração e cuidados com o original 26
20. Bases de atividade e resultados conferidos 27
21. Laboratório: seguir os dados no código 28
22. Mapa de IDs e vocabulário 29
23. Exercícios SQL: orientações de entrega 31
Exercícios • A — Estrutura e integridade 32
Exercícios • B — Consultas básicas 33
Exercícios • C — Relacionamentos 34
Exercícios • D — Agregações e subconsultas 35
Exercícios • E — Inserções, alterações e exclusões 36
Exercícios • F — Desafios integradores 37

1. Arquitetura: quem faz o quê?

HTML descreve a estrutura da página; CSS cuida da apresentação; JavaScript responde às ações e modifica a página. SQL descreve operações sobre os dados. No projeto, o JavaScript conecta essas partes por meio da biblioteca sql.js.

Arquivo	Responsabilidade
index.html	Formulários, campos, botões, abas e locais para mostrar resultados.
app.js	Eventos, leitura de campos, mensagens e atualização visual.
db.js	Inicialização do SQLite e métodos de acesso por entidade.
DDL.sql	Criação de tabelas e restrições.
DML.sql	Carga de dados, atualizações e exclusão de exemplo.
DQL.sql	Quinze consultas de leitura.



O banco dentro do navegador

sql.js é uma distribuição do SQLite compilada para WebAssembly, ou WASM. A biblioteca JavaScript carrega o motor e permite executar SQL em uma instância de banco em memória. As chamadas de cadastro não enviam os registros para uma API.

A biblioteca e seu arquivo WASM são obtidos por CDN. Isso é tráfego de rede para carregar uma dependência; não significa que os dados digitados sejam enviados a um servidor de banco neste código.

Comparação com uma aplicação que usa API

Exemplo didático • adaptação

```
Navegador: formulário → JavaScript → fetch()
Servidor: API → validação → comando parametrizado → banco
Retorno: banco → API → JSON → JavaScript → DOM
```

Em uma aplicação centralizada, a API controla autenticação, autorização e validação no servidor. Credenciais do banco ficam no servidor. O formulário e a atualização do DOM continuam seguindo os princípios estudados aqui.

2. Preparação e ordem de carregamento

Organização esperada pelas referências do HTML

```
universidade/
  index.html
  css/style.css       (referenciado, mas não enviado)
  js/db.js
  js/app.js
  DDL.sql
  DML.sql
  DQL.sql
```

O index.html referencia css/style.css, mas esse arquivo não está entre os anexos. A ausência interfere na aparência e pode deixar abas e mensagens sem o comportamento visual esperado. Os exemplos desta apostila se baseiam na estrutura e no código disponíveis, sem inventar o CSS original.

Recorte original • index.html • linhas 378–380

```
378  <script src="https://cdnjs.cloudflare.com/ajax/libs/sql.js/1.10.2/sql-
     ↳ wasm.js"></script>
379  <script src="js/db.js"></script>
380  <script src="js/app.js"></script>
```

↳ continuação visual da mesma linha do arquivo original.

A ordem é importante: primeiro a biblioteca define initSqlJs; depois db.js define DB; por último app.js utiliza DB. Os scripts estão no final do body e não usam async, preservando essa ordem de execução.

Executando localmente

Exemplo didático • adaptação

```
python -m http.server 8000
# Abra http://localhost:8000 no navegador.
```

Execute o comando na pasta do projeto. Outra opção é usar uma extensão de servidor local do editor. O acesso à CDN precisa estar disponível, a menos que você adapte as dependências para arquivos locais.

Preparando as tabelas e os registros

- Abra a página e aguarde o status do motor SQLite.
- Clique em Carregar Script e selecione DDL.sql.
- Carregue DML.sql para inserir e alterar os dados de exemplo.
- Use as abas e depois carregue DQL.sql para visualizar as consultas.

DDL é destrutivo na base existente
O script contém DROP TABLE. Executá-lo novamente remove as tabelas anteriores antes de recriá-las. Para restaurar uma atividade, execute DDL e depois DML conscientemente.

3. DOM: encontrando o campo certo

O DOM é a representação do documento HTML como objetos. Um input é um elemento DOM; o texto digitado é uma propriedade desse elemento. O JavaScript precisa primeiro encontrar o elemento e depois ler seu valor.

Recorte original • index.html • linhas 83–89

```
  <label for="p-documento">Documento *</label>
  <input type="text" id="p-documento" placeholder="CPF ou RG" required />
</div>
<div class="form-group span-2">
  <label for="p-nome">Nome Completo *</label>
  <input type="text" id="p-nome" placeholder="Nome completo" required />
</div>
```

Exemplo didático • adaptação

```
const campo = document.getElementById('p-nome');
const nome = campo.value;

// Se o usuário digitou Joana:
console.log(nome); // "Joana"
```

Expressão ou atributo	Significado
id="p-nome"	Identificador do elemento. Deve ser único no documento.
for="p-nome"	Associa o label ao input com esse ID.
getElementById(...)	Retorna o elemento encontrado, ou null se não existir.
campo.value	Lê ou escreve o conteúdo do campo.
campo.textContent	Não é o mecanismo para ler o texto digitado em um input.

O texto do label pode mudar sem alterar a captura, desde que o ID permaneça correto. Por outro lado, trocar p-nome por outro ID no HTML exige atualizar todas as referências correspondentes no JavaScript.

ID não é o mesmo que name

name identifica campos em mecanismos como envio convencional de formulários e FormData. Este projeto lê valores diretamente pelos IDs; por isso não depende de name nesses campos. A propriedade nome do objeto JavaScript também não precisa ter o mesmo texto do ID.

Exemplo didático • adaptação

```
const dados = {
  nome: document.getElementById('p-nome').value
};
// p-nome: ID no HTML.
// nome: propriedade do objeto que será entregue ao método do banco.
```

Para verificar
Se getElementById retornar null, confira o ID e o momento de execução. Tentar acessar .value de null gera um erro antes de chegar ao banco.

4. Inicialização, eventos e envio

Recorte original • index.html • linhas 24–29

```
<section id="status-bar">
  <span id="db-status" class="badge badge-warning">⏳ Carregando banco...</span>
  <button id="btn-init-db" disabled>▶️ Inicializar Banco</button>
  <button id="btn-clear-db" class="btn-danger" disabled>🗑️ Limpar Banco</button>
  <button id="btn-export-db" disabled>💾 Exportar .db</button>
</section>
```

Recorte original • app.js • linhas 98–113

```
document.addEventListener('DOMContentLoaded', async () => {

  const statusEl = document.getElementById('db-status');
  const btnInit = document.getElementById('btn-init-db');
  const btnClear = document.getElementById('btn-clear-db');
  const btnExport = document.getElementById('btn-export-db');

  // ── Carrega sql.js WASM e cria o banco ──────────────────
  try {
    await DB.init();
    statusEl.textContent = '✅ SQLite pronto (aguardando DDL)';
    statusEl.className = 'badge badge-success';
    btnInit.disabled = false;
    btnClear.disabled = false;
    btnExport.disabled = false;
    showToast('Motor SQLite pronto! Crie as tabelas via DDL ou Console.', 'info');
```

DOMContentLoaded indica que o documento foi analisado. O callback localiza os elementos e inicia o banco. await espera a conclusão de DB.init() antes de atualizar o status e habilitar os botões indicados.

disabled é uma propriedade booleana do botão. textContent troca o texto do span; className altera as classes usadas para apresentação. A aparência dessas classes depende do CSS.

O evento do formulário

Recorte original • app.js • linhas 265–269

```
document.getElementById('form-pessoa').addEventListener('submit', e => {
  e.preventDefault();
  try {
    const codVal = document.getElementById('p-codigo').value;
    DB.Pessoa.inserir({
```

addEventListener registra a função que será chamada quando ocorrer submit. e é o objeto do evento. preventDefault impede a ação padrão de envio e navegação, permitindo que o JavaScript processe os dados.

Recorte original • index.html • linhas 127–132

```
      <div class="btn-row">
        <button type="submit">💾 Inserir Pessoa</button>
        <button type="button" id="btn-list-pessoa" class="btn-secondary">🔍 Listar
    ↳ Todas</button>
      </div>
    </form>
</div>
```

↳ continuação visual da mesma linha do arquivo original.

O botão submit envia o formulário. O botão type="button" permite listar sem enviar. Escutar submit também contempla o envio por Enter. No envio normal pelo usuário, as restrições HTML, como required, são verificadas pelo navegador antes do evento submit.

Limite atual
O código habilita os botões de controle após a inicialização, mas não bloqueia todos os formulários enquanto o motor carrega ou enquanto faltam tabelas. Uma evolução é liberar cadastros somente quando a estrutura estiver pronta.

5. Captura de pessoa: HTML → objeto

O formulário form-pessoa agrupa dados pessoais e endereço. O cadastro extrai cada campo pelo seu ID e reúne os valores em um objeto passado a DB.Pessoa.inserir().

Recorte original • index.html • linhas 74–82

```
74  <form id="form-pessoa">
75    <fieldset>
76      <legend>Dados Pessoais</legend>
77      <div class="form-grid">
78        <div class="form-group">
79          <label for="p-codigo">Código (opcional)</label>
80          <input type="number" id="p-codigo" placeholder="Auto ou informe nº" />
81        </div>
82        <div class="form-group">
```

Recorte original • index.html • linhas 100–106

```
100  <div class="form-group">
101    <label for="p-numero">Número *</label>
102    <input type="text" id="p-numero" placeholder="Nº" required />
103  </div>
104  <div class="form-group">
105    <label for="p-complemento">Complemento</label>
106    <input type="text" id="p-complemento" placeholder="Apto, Bloco..." />
```

Recorte original • index.html • linhas 116–123

```
116  <div class="form-group">
117    <label for="p-estado">Estado (UF) *</label>
118    <input type="text" id="p-estado" maxlength="2" placeholder="SC, SP, etc." required
     ↳ />
119  </div>
120  <div class="form-group">
121    <label for="p-cep">CEP *</label>
122    <input type="text" id="p-cep" placeholder="00000-000" required />
123  </div>
```

↳ continuação visual da mesma linha do arquivo original.

Onde a captura acontece

Recorte original • app.js • linhas 269–279

```
269  DB.Pessoa.inserir({
270    codigo:      codVal ? parseInt(codVal) : null,
271    documento:   document.getElementById('p-documento').value,
272    nome:        document.getElementById('p-nome').value,
273    logradouro:  document.getElementById('p-logradouro').value,
274    numero:      document.getElementById('p-numero').value,
275    complemento: document.getElementById('p-complemento').value || null,
276    bairro:      document.getElementById('p-bairro').value,
277    cidade:      document.getElementById('p-cidade').value,
278    estado:      document.getElementById('p-estado').value.toUpperCase(),
279    cep:         document.getElementById('p-cep').value,
```

Cada expressão à direita de dois-pontos fornece o valor da propriedade à esquerda. documento recebe o conteúdo de p-documento; nome recebe p-nome; estado recebe p-estado convertido para maiúsculas. O método do banco conhece propriedades como dados.nome e dados.estado, sem precisar conhecer o DOM.

Separação de responsabilidades
app.js conhece os IDs da página. db.js conhece tabelas e colunas. O objeto dados é a ponte entre essas duas partes.

Rastreamento de uma informação

Exemplo didático • adaptação

```
HTML:       <input id="p-nome" ... />
Usuário:    Joana Pereira
app.js:     .value → "Joana Pereira"
Objeto:     { nome: "Joana Pereira", ... }
db.js:      dados.nome
SQL:        parâmetro da coluna nome
PESSOA:     linha armazenada com esse nome
```

O endereço é composto por vários campos do formulário e por várias colunas da tabela. O número do endereço é texto: isso permite valores como S/N ou 120-A.

6. Tipos, conversões e validações

A propriedade value de um input retorna uma string, inclusive quando type é number. Antes de usar um valor como código, semestre, ano, carga horária ou nota, o código faz conversões.

Entrada	Conversão no projeto	Exemplo de resultado
Código opcional	codVal ? parseInt(codVal) : null	"7" → 7; "" → null
Estado	.toUpperCase()	"sc" → "SC"
Complemento	.value || null	"" → null
Carga horária	parseInt(...)	"60" → 60
Nota	nota !== "" ? parseFloat(nota) : null	"0" → 0; "" → null

Vazio não é zero

null representa ausência de valor. Uma nota zero é uma avaliação registrada. Usar numero || null depois da conversão numérica apagaria o zero, porque zero é falsy. O teste explícito de string vazia usado para nota preserva essa diferença.

Exemplo didático • adaptação

```
const texto = document.getElementById('m-nota').value;
const nota = texto !== '' ? parseFloat(texto) : null;
// "" → null; "0" → 0; "8.5" → 8.5
```

Validação mais rigorosa

Exemplo didático • adaptação

```
const texto = document.getElementById('d-ch').value.trim();
const carga = Number(texto);

if (texto === '' || !Number.isInteger(carga) || carga <= 0) {
  throw new Error('Informe uma carga horária inteira e positiva.');
}
```

parseInt e parseFloat podem aceitar um prefixo numérico de texto inválido. Number exige conversão do conteúdo completo, mas Number("") produz zero; por isso a checagem do vazio continua necessária. Também valide NaN e valores não finitos quando aplicável.

Três níveis complementares

O HTML orienta a entrada com required, min, max e step. O JavaScript normaliza e verifica regras. O banco protege integridade com NOT NULL, UNIQUE, CHECK e chaves. Em aplicações com API, o servidor também precisa validar, pois não pode confiar nas verificações do navegador.

O que as restrições não garantem
NOT NULL aceita texto vazio; duas letras não garantem uma UF real; UNIQUE não valida CPF; TEXT em data_admissao não comprova data válida. As regras de negócio precisam ser definidas explicitamente.

7. INSERT: objeto → parâmetros → banco

O método Pessoa.inserir possui dois caminhos: com código manual, inclui codigo no INSERT; sem código, omite essa coluna para que o SQLite gere o identificador.

Recorte original • db.js • linhas 115–125

```
115  const sql = `
116    INSERT INTO PESSOA
117      (documento, nome, logradouro, numero, complemento, bairro, cidade, estado, cep)
118    VALUES
119      (?, ?, ?, ?, ?, ?, ?, ?, ?)
120  `;
121  return run(sql, [
122    dados.documento, dados.nome,
123    dados.logradouro, dados.numero, dados.complemento,
124    dados.bairro, dados.cidade, dados.estado, dados.cep
125  ]);
```

A lista de colunas determina a ordem dos valores. Cada ? recebe o item da mesma posição no array. Se você trocar a ordem de nome e documento, poderá armazenar dados nas colunas erradas ou provocar uma restrição.

Marcador	Coluna	Propriedade
1	documento	dados.documento
2	nome	dados.nome
3	logradouro	dados.logradouro
4	numero	dados.numero
5	complemento	dados.complemento
6	bairro	dados.bairro
7	cidade	dados.cidade
8	estado	dados.estado
9	cep	dados.cep

Por que parametrizar?

Exemplo didático • adaptação

```
// Evite concatenar o texto do usuário ao comando:
// "SELECT * FROM PESSOA WHERE nome = '" + nome + "'"

// Separe estrutura SQL e valores:
const rows = DB.query(
  'SELECT * FROM PESSOA WHERE nome = ?',
  [nome]
);
```

Parâmetros tratam os valores como dados, preservam textos com apóstrofos e reduzem o risco de injeção SQL. Eles não substituem validação e não servem para substituir livremente nomes de tabelas ou colunas. Identificadores dinâmicos exigem uma lista controlada de opções.

8. Dentro de DB: memória e execução

Recorte original • db.js • linhas 34–43

```
34  async function init() {
35    _SQL = await initSqlJs({
36      locateFile: file => `https://cdnjs.cloudflare.com/ajax/libs/sql.js/1.10.2/${file}`
37    });
38
39    _db = new _SQL.Database();   // banco vazio em memória
40    _db.run("PRAGMA foreign_keys = ON;"); // Habilita suporte a chaves estrangeiras
41
42    return _db;
43  }
```

initSqlJs carrega a biblioteca e o WASM. new _SQL.Database() cria uma instância vazia em memória. PRAGMA foreign_keys = ON ativa a fiscalização das chaves estrangeiras nessa conexão.

As tabelas serão criadas quando o usuário executar o DDL.

Encapsulamento

Exemplo didático • adaptação

```
const DB = (() => {
  let _db = null;
  // Funções internas acessam _db.
  return { init, run, query /* e demais métodos */ };
})();
```

A função imediatamente executada cria um escopo interno. O código da interface usa a API pública de DB, enquanto _db permanece interno. Isso centraliza o acesso ao motor e facilita organizar as operações por entidade.

Recorte original • db.js • linhas 49–54

```
49  // ──────────────────────────────────────────────────────────
50  function run(sql, params = []) {
51    if (!_db) throw new Error('Banco não inicializado. Chame DB.init() primeiro.');
52    _db.run(sql, params);
53    return _db.getRowsModified();   // nº de linhas afetadas
54  }
```

run executa a instrução e retorna getRowsModified(). Para INSERT, UPDATE e DELETE, esse retorno permite verificar a quantidade de linhas afetadas pela última instrução de alteração. Ele não é a soma de todas as instruções de um script nem o identificador gerado.

Persistência e exportação

Recarregar a página perde a instância em memória. Os botões Inicializar Banco e Limpar Banco chamam novamente init, criando um banco vazio. Exportar chama exportDB(), gera bytes, cria um Blob e aciona o download de universidade.db.

Nome correto do método
A API pública expõe DB.exportDB(), e esse é o nome usado por app.js. O comentário inicial de db.js menciona DB.export(), mas esse nome não está exposto na implementação.

O projeto não oferece importação do arquivo .db. Baixar uma cópia não faz os próximos cadastros serem salvos nela automaticamente. Uma versão com persistência precisaria implementar um mecanismo adicional, como armazenamento local apropriado ou uma API.

9. SELECT: linhas → objetos JavaScript

Recorte original • db.js • linhas 129–135

```
129  listarTodos() {
130    return query(`SELECT * FROM PESSOA ORDER BY nome`);
131  },
132
133  buscarPorId(codigo) {
134    return query(`SELECT * FROM PESSOA WHERE codigo = ?`, [codigo]);
135  },
```

listarTodos executa uma consulta e devolve um array de objetos. buscarPorId também retorna um array: consultar um código único não transforma automaticamente o resultado em um objeto isolado.

Recorte original • db.js • linhas 60–73

```
60  function query(sql, params = []) {
61    if (!_db) throw new Error('Banco não inicializado. Chame DB.init() primeiro.');
62
63    const stmt    = _db.prepare(sql);
64    const results = [];
65
66    stmt.bind(params);
67    while (stmt.step()) {
68      results.push(stmt.getAsObject());
69    }
70    stmt.free();
71
72    return results;
73  }
```

Etapa	O que acontece
prepare(sql)	Prepara a instrução.
bind(params)	Associa os valores aos parâmetros.
step()	Avança para a próxima linha disponível.
getAsObject()	Transforma a linha atual em objeto.
results.push(...)	Adiciona o objeto à lista.
free()	Libera os recursos da instrução.

Exemplo de formato • trecho ilustrativo do resultado

```
// Consulta:
SELECT codigo, nome FROM PESSOA ORDER BY nome;

// Formato do retorno em JavaScript:
[
  { codigo: 4, nome: "Beatriz Lima Souza" },
  { codigo: 3, nome: "Lucas Gabriel Silveira" }
  // demais linhas...
]
```

Os nomes das colunas tornam-se nomes de propriedades. SELECT p.nome AS estudante produz a propriedade estudante. Se não houver linhas, o retorno será []. Ao combinar tabelas, use aliases

para evitar nomes repetidos no mesmo objeto.

Melhoria de liberação
A implementação chama stmt.free() no caminho normal. Um try/finally ao redor do processamento garantiria essa liberação também se bind ou o processamento das linhas lançar um erro.

10. Onde o retorno entra no HTML?

Recorte original • index.html • linhas 134–139

```
134  <div class="card">
135    <h2>📋 Lista de Pessoas</h2>
136    <div id="table-pessoa" class="table-wrapper">
137      <p class="placeholder-msg">Clique em "Listar Todas" para carregar.</p>
138    </div>
139  </div>
```

O elemento table-pessoa é um contêiner de resultado. Apesar do nome, ele é uma div, não uma table. O parágrafo inicial funciona como mensagem de espera. O JavaScript substituirá o conteúdo dessa div pela tabela montada.

Recorte original • app.js • linhas 280–287

```
280      });
281      e.target.reset();
282      showToast('Pessoa inserida com sucesso!', 'success');
283      renderTable('table-pessoa', DB.Pessoa.listarTodos(), 'codigo');
284    } catch (err) {
285      showToast(`Erro: ${err.message}`, 'error');
286    }
287  });
```

Após inserir, e.target.reset() limpa os campos do formulário. showToast mostra uma mensagem. Em seguida, listarTodos consulta o banco novamente e renderTable recebe o ID do destino, os registros e a coluna usada para ações.

Recorte original • app.js • linhas 65–67

```
65  function renderTable(containerId, rows, idCol = null) {
66    document.getElementById(containerId).innerHTML = buildTable(rows, idCol);
67  }
```

Leia a chamada de dentro para fora

Exemplo didático • expansão da chamada original

```
const pessoas = DB.Pessoa.listarTodos();
const htmlTabela = buildTable(pessoas, 'codigo');
const destino = document.getElementById('table-pessoa');
destino.innerHTML = htmlTabela;
```

innerHTML substitui os elementos filhos do destino, mas preserva a própria div e seu ID. A atualização visual ocorre porque o código escreve no DOM. Executar apenas INSERT ou SELECT não redesenha a página automaticamente.

Entrada / saída	Elemento utilizado	Operação
Entrada do nome	input #p-nome	Leitura de .value

Entrada / saída	Elemento utilizado	Operação
Mensagem de status	span #db-status	Escrita em .textContent
Retorno das pessoas	div #table-pessoa	Escrita em .innerHTML
Campos para edição	inputs do formulário	Escrita em .value

E se o banco mudou e a tela não?
Verifique se uma nova consulta foi feita e se o ID de destino corresponde à listagem correta. O projeto não possui um mecanismo automático de sincronização de todas as abas.

11. Montando uma tabela dinâmica

Recorte original • app.js • linhas 36–47

```
36  function buildTable(rows, idCol = null) {
37    if (!rows || rows.length === 0) {
38      return `<p class="placeholder-msg">Nenhum registro encontrado.</p>`;
39    }
40
41    const cols = Object.keys(rows[0]);
42
43    const thead = `<thead><tr>${cols.map(c => `<th>${c}</th>`).join('')}${idCol ?
    ↳ '<th>Ações</th>' : ''}</tr></thead>`;
44
45    const tbody = rows.map(row => {
46      const cells = cols.map(c => `<td>${row[c] ?? ''}</td>`).join('');
47      const actions = idCol
```

↳ continuação visual da mesma linha do arquivo original.

Quando rows está vazio, buildTable retorna uma mensagem. Caso contrário, Object.keys(rows[0]) recupera os nomes das propriedades da primeira linha. O código usa esses nomes para gerar os cabeçalhos e as células de todas as linhas.

Exemplo didático • adaptação

```
const row = { codigo: 3, nome: 'Lucas', nota: 0 };
const cols = Object.keys(row);
// ['codigo', 'nome', 'nota']

cols.map(c => `<td>${row[c] ?? ''}</td>`).join('');
// <td>3</td><td>Lucas</td><td>0</td>
```

map transforma cada item em outro valor. join junta os fragmentos em uma string. row[c] lê a propriedade cujo nome está em c. O operador ?? usa a alternativa apenas para null ou undefined, preservando zero.

Atenção: conteúdo do banco em innerHTML

Uma string com marcação pode ser interpretada pelo navegador se for inserida com innerHTML. Parametrizar SQL não resolve esse problema de apresentação. Para valores do usuário, prefira elementos DOM preenchidos com textContent.

Extensão proposta • listagem segura de duas colunas

```
function mostrarPessoasSeguras(rows) {
  const destino = document.getElementById('table-pessoa');
  destino.replaceChildren();
  if (rows.length === 0) {
    destino.textContent = 'Nenhum registro encontrado.';
    return;
  }
  const tabela = document.createElement('table');
  const topo = tabela.createTHead().insertRow();
  for (const titulo of ['Código', 'Nome']) {
    const th = document.createElement('th');
    th.textContent = titulo;
    topo.appendChild(th);
  }
  const corpo = tabela.createTBody();
  for (const pessoa of rows) {
    const linha = corpo.insertRow();
    linha.insertCell().textContent = String(pessoa.codigo);
    linha.insertCell().textContent = pessoa.nome;
  }
  destino.appendChild(tabela);
}
```

A função é uma alternativa didática para listar pessoas. Para usá-la, chame mostrarPessoasSeguras(DB.Pessoa.listarTodos()). Ela não implementa os botões de exclusão da tabela original.

12. Caso completo: disciplina

Recorte original • index.html • linhas 228–249

```
228    <form id="form-disciplina">
229      <div class="form-grid">
230        <div class="form-group">
231          <label for="d-codigo">Código da Disciplina (PK) *</label>
232          <input type="text" id="d-codigo" placeholder="Ex: CC001" required />
233        </div>
234        <div class="form-group span-2">
235          <label for="d-nome">Nome da Disciplina *</label>
236          <input type="text" id="d-nome" placeholder="Ex: Banco de Dados I" required />
237        </div>
238        <div class="form-group">
239          <label for="d-ch">Carga Horária (h) *</label>
240          <input type="number" id="d-ch" placeholder="Ex: 60" min="1" required />
241        </div>
242      </div>
243      <div class="btn-row">
244        <button type="submit">□ Inserir Disciplina</button>
245        <button type="button" id="btn-list-disciplina" class="btn-secondary">□ Listar
     ↳ Todas</button>
246      </div>
247    </form>
248  </div>
249
```

↳ continuação visual da mesma linha do arquivo original.

O formulário é identificado por form-disciplina; os campos são d-codigo, d-nome e d-ch. O botão de listagem tem seu próprio ID e não envia o formulário.

Recorte original • app.js • linhas 351–365

```
351  document.getElementById('form-disciplina').addEventListener('submit', e => {
352    e.preventDefault();
353    try {
354      DB.Disciplina.inserir({
355        codigo:        document.getElementById('d-codigo').value.trim(),
356        nome:          document.getElementById('d-nome').value,
357        carga_horaria: parseInt(document.getElementById('d-ch').value),
358      });
359      e.target.reset();
360      showToast('Disciplina inserida com sucesso!', 'success');
361      renderTable('table-disciplina', DB.Disciplina.listarTodas(), 'codigo');
362    } catch (err) {
363      showToast(`Erro: ${err.message}`, 'error');
364    }
365  });
```

O código remove espaços das extremidades do código, lê o nome e converte a carga horária para inteiro. O objeto resultante é passado ao método inserir. Depois, o formulário é limpo e a listagem é refeita.

Onde a disciplina é armazenada

db.js • método Disciplina.inserir, transcrito da implementação

```
inserir(dados) {
  return run(
    `INSERT INTO DISCIPLINA (codigo, nome, carga_horaria)
     VALUES (?, ?, ?)`,
    [dados.codigo, dados.nome, dados.carga_horaria]
  );
}
```

Recorte original • index.html • linhas 250–255

```
250  <div class="card">
251    <h2>□ Lista de Disciplinas</h2>
252    <div id="table-disciplina" class="table-wrapper">
253      <p class="placeholder-msg">Clique em "Listar Todas" para carregar.</p>
254    </div>
255  </div>
```

A div table-disciplina recebe o retorno. Não confunda esse ID com o nome DISCIPLINA da tabela SQL: um identifica um elemento na página; o outro identifica uma estrutura no banco.

Passo	Exemplo
Usuário preenche	WEB01; Desenvolvimento Web; 80
Objeto	{ codigo: "WEB01", nome: "Desenvolvimento Web", carga_horaria: 80 }
INSERT	Três colunas e três parâmetros.
SELECT	DB.Disciplina.listarTodas() consulta o banco.
DOM	renderTable escreve em #table-disciplina.

Prática guiada
Cadastre WEB01 uma vez e tente repetir. Explique por que a primeira tentativa insere e a segunda viola a chave. Para testar o cadastro, não use o botão de exclusão original de disciplina antes de estudar seus problemas no capítulo 19.

13. Estudantes, matrículas e orientação

Um estudante precisa de uma pessoa existente

Recorte original • index.html • linhas 148–165

```
148  <form id="form-estudante">
149    <div class="form-grid">
150      <div class="form-group">
151        <label for="e-codigo">Código da Pessoa (PK/FK) *</label>
152        <input type="number" id="e-codigo" placeholder="Código cadastrado em Pessoa"
     ↳ required />
153      </div>
154      <div class="form-group">
155        <label for="e-matricula">Nº Matrícula (UQ) *</label>
156        <input type="text" id="e-matricula" placeholder="Ex: MAT-202401" required />
157      </div>
158      <div class="form-group">
159        <label for="e-curso">Curso *</label>
160        <input type="text" id="e-curso" placeholder="Ex: Ciência da Computação" required
     ↳  />
161      </div>
162      <div class="form-group">
163        <label for="e-orientador">Cód. Orientador (Funcionário)</label>
164        <input type="number" id="e-orientador" placeholder="Código do docente
     ↳ (opcional)" />
165      </div>
```

↳ continuação visual da mesma linha do arquivo original.

e-codigo não gera uma pessoa automaticamente. Seu valor precisa existir em PESSOA. O banco reaproveita o código como chave primária de ESTUDANTE e também como chave estrangeira para PESSOA.

Nota opcional na matrícula

Recorte original • index.html • linhas 287–297

```
287      <div class="btn-row">
288        <button type="submit">▯ Matricular</button>
289        <button type="button" id="btn-list-matricula" class="btn-secondary">▯ Listar
     ↳ Todas</button>
290      </div>
291    </form>
292  </div>
293
294  <div class="card">
295    <h2>▯ Matrículas em Disciplinas</h2>
296    <div id="table-matricula" class="table-wrapper">
297      <p class="placeholder-msg">Clique em "Listar Todas" para carregar.</p>
```

↳ continuação visual da mesma linha do arquivo original.

Recorte original • app.js • linhas 376–386

```
376  document.getElementById('form-matricula').addEventListener('submit', e => {
377    e.preventDefault();
378    try {
379      const nota = document.getElementById('m-nota').value;
380      DB.MatriculaDisciplina.inserir({
381        id_estudante:      parseInt(document.getElementById('m-estudante-id').value),
382        disciplina_codigo: document.getElementById('m-disciplina-codigo').value.trim(),
383        semestre:          parseInt(document.getElementById('m-semestre').value),
384        ano:               parseInt(document.getElementById('m-ano').value),
385        nota_final:        nota !== '' ? parseFloat(nota) : null,
386      });
```

Os IDs mostram a origem de cada parte da matrícula. Código de disciplina permanece texto; semestre, ano e estudante são convertidos para inteiros; a nota é convertida para decimal ou null. A chave composta identifica uma oferta cursada em um período.

Orientação usa UPDATE

Recorte original • index.html • linhas 343–352

```
343  <form id="form-orientacao">
344    <div class="form-grid">
345      <div class="form-group">
346        <label for="o-orientado-id">Código do Estudante (Orientado) *</label>
347        <input type="number" id="o-orientado-id" placeholder="Código do estudante"
     ↳ required />
348      </div>
349      <div class="form-group">
350        <label for="o-orientador-id">Código do Funcionário (Orientador) *</label>
351        <input type="number" id="o-orientador-id" placeholder="Código do funcionário"
     ↳ required />
352      </div>
```

↳ continuação visual da mesma linha do arquivo original.

Exemplo didático • equivalência entre formulário e comando

```
DB.Orientacao.definir({
  id_funcionario: 1,
  id_estudante: 6
});

// Em db.js:
UPDATE ESTUDANTE SET id_orientador = ? WHERE codigo = ?
// Parâmetros: [1, 6]
```

Não existe uma tabela ORIENTACAO no DDL. O relacionamento é representado pelo atributo id_orientador de ESTUDANTE, que referencia FUNCIONARIO. Remover uma orientação significa atribuir NULL ao atributo, preservando o estudante.

14. Do banco para campos de edição

O retorno pode preencher uma tabela ou um formulário. Para editar uma pessoa, consulte a linha por código, recupere o objeto e atribua seus valores à propriedade value dos inputs.

Extensão proposta • carregamento de pessoa para edição

```
let codigoEmEdicao = null;

function carregarPessoaParaEdicao(codigo) {
  const pessoa = DB.Pessoa.buscarPorId(codigo)[0];
  if (!pessoa) {
    showToast('Pessoa não encontrada.', 'error');
    return;
  }
  codigoEmEdicao = pessoa.codigo;
  const campos = {
    'p-codigo': 'codigo',
    'p-documento': 'documento',
    'p-nome': 'nome',
    'p-logradouro': 'logradouro',
    'p-numero': 'numero',
    'p-complemento': 'complemento',
    'p-bairro': 'bairro',
    'p-cidade': 'cidade',
    'p-estado': 'estado',
    'p-cep': 'cep'
  };
  for (const [id, coluna] of Object.entries(campos)) {
    document.getElementById(id).value = pessoa[coluna] ?? '';
  }
  document.getElementById('p-codigo').readOnly = true;
}
```

O [0] obtém a primeira linha do array. O teste de existência evita tentar ler propriedades de undefined. O mapa relaciona IDs e colunas, reduzindo repetição. ?? converte a ausência de complemento em campo visualmente vazio.

Salvar precisa distinguir inserir e atualizar

Extensão proposta • fragmento para integração no evento existente

```
// No lugar da chamada fixa a inserir, dentro do listener existente:
if (codigoEmEdicao === null) {
  DB.Pessoa.inserir(dados);
} else {
  const afetadas = DB.Pessoa.atualizar(codigoEmEdicao, dados);
  if (afetadas === 0) throw new Error('Pessoa não encontrada.');
}

codigoEmEdicao = null;
document.getElementById('p-codigo').readOnly = false;
document.getElementById('form-pessoa').reset();
mostrarPessoasSeguras(DB.Pessoa.listarTodos());
```

Nesse fragmento, dados deve ser o objeto completo de pessoa, construído com os mesmos campos do cadastro. É necessário guardar esse objeto em uma variável em vez de passá-lo diretamente a inserir. Não adicione um segundo listener submit: adapte o atual.

Comportamento original
O projeto já possui Pessoa.buscarPorId e Pessoa.atualizar em db.js, mas a interface enviada não oferece esse fluxo completo. O formulário original sempre chama inserir, mesmo se estiver preenchido com um código existente.

15. UPDATE, DELETE e sincronização

UPDATE com filtro

Exemplo didático • adaptação

```
UPDATE MATRICULA_DISCIPLINA
SET nota_final = 8.7
WHERE id_estudante = 3
  AND disciplina_codigo = 'CC001'
  AND semestre = 1
  AND ano = 2024;
```

SET define as novas informações e WHERE seleciona os registros. Sem WHERE, todas as linhas da tabela são candidatas à alteração. Na matrícula, a chave completa evita atingir outros períodos do mesmo estudante.

Comando executado sem erro não significa registro encontrado. Um UPDATE com código inexistente pode afetar zero linhas. Verifique o retorno antes de exibir uma mensagem que afirme alteração efetiva.

DELETE e chaves estrangeiras

Exemplo didático • adaptação

```
DELETE FROM PESSOA WHERE codigo = ?;

DELETE FROM MATRICULA_DISCIPLINA
WHERE id_estudante = ? AND disciplina_codigo = ?
  AND semestre = ? AND ano = ?;
```

Regra do DDL	Efeito
ON DELETE CASCADE	Propaga a exclusão para linhas dependentes.
ON DELETE SET NULL	Preserva a linha dependente e remove sua referência.
Pessoa → subclasses	Excluir uma pessoa remove seus registros de estudante e funcionário.
Estudante → matrículas	Excluir o estudante remove suas matrículas.
Funcionário → orientandos	Excluir o funcionário deixa os estudantes sem esse orientador.

Transações

Extensão proposta • duas inserções como uma operação

```
DB.run('BEGIN TRANSACTION');
try {
  DB.Pessoa.inserir(dadosPessoa);
  DB.Estudante.inserir(dadosEstudante);
  DB.run('COMMIT');
} catch (err) {
  DB.run('ROLLBACK');
  throw err;
}
```

O exemplo pressupõe que os objetos foram validados e que o código do estudante corresponde ao da pessoa inserida. A transação evita deixar uma pessoa parcialmente cadastrada quando o segundo passo falha. Esse controle não está implementado nos formulários originais separados.

Após confirmar a alteração, faça uma nova consulta e atualize as listagens afetadas. Remover apenas uma tr da tela não sincroniza as outras abas nem revela todos os efeitos de uma cascata.

16. Arquivos SQL e console da página

Recorte original • index.html • linhas 34–37

```
34  <section class="card" id="sql-console">
35    <h2>□□ Console SQL</h2>
36    <textarea id="sql-input"
37      placeholder="Digite ou cole um comando SQL aqui...&#10;Exemplo: SELECT * FROM
    ↳ Pessoa;"></textarea>
```

↳ continuação visual da mesma linha do arquivo original.

Recorte original • index.html • linhas 44–52

```
44    <div style="display: flex; gap: 8px; align-items: center;">
45      <button id="btn-load-script" class="btn-danger">□ Carregar Script</button>
46      <input type="file" id="script-file-picker" accept=".sql" style="display: none;" />
47    </div>
48  </div>
49
50  <div id="sql-result-box">
51    <p class="placeholder-msg">Os resultados aparecerão aqui.</p>
52  </div>
```

sql-input é a origem do comando digitado. sql-result-box é o destino dos resultados. script-file-picker recebe um arquivo escolhido pelo usuário; btn-load-script abre esse seletor por código.

app.js • fluxo de leitura do arquivo, recorte lógico

```
const reader = new FileReader();
reader.onload = ev => {
  executarConteudoSQL(ev.target.result, file.name);
};
reader.readAsText(file);
```

FileReader lê o arquivo como texto. executarConteudoSQL copia esse texto para o textarea e o passa a DB.executeScript. Selecionar um arquivo não equivale a enviar seu conteúdo a um servidor neste código.

Dois formatos de retorno

Exemplo didático • adaptação

```
// DB.query():
[{ codigo: 3, nome: 'Lucas' }]

// DB.executeScript(), baseado em _db.exec():
[{ columns: ['codigo', 'nome'], values: [[3, 'Lucas']] }]
```

O script pode produzir vários conjuntos de resultados. app.js percorre columns e values, constrói objetos e chama buildTable para cada resultado. Os nomes dos cabeçalhos vêm das colunas ou aliases da consulta.

Limite da detecção do console

O botão Executar SQL usa /^\s*SELECT/i para escolher o caminho de leitura. Consultas iniciadas por comentário ou WITH não são reconhecidas por esse teste. Para os arquivos completos enviados, use Carregar Script. No console, execute uma consulta individual começando diretamente por SELECT.

Script não é automaticamente atômico
executeScript não envolve todos os comandos em uma transação explícita. Se houver erro no meio, comandos anteriores podem já ter produzido efeitos. Planeje BEGIN/COMMIT/ROLLBACK quando precisar garantir uma unidade de trabalho.

17. Modelo relacional e regras do DDL

Tabela	Chave principal	Papel
PESSOA	codigo	Dados pessoais e endereço.
FUNCIONARIO	codigo: PK e FK	Registro funcional, admissão e setor.
ESTUDANTE	codigo: PK e FK	Matrícula, curso e orientador opcional.
DISCIPLINA	codigo textual	Nome e carga horária.
MATRICULA_DISCIPLINA	estudante + disciplina + semestre + ano	Histórico e nota opcional.
MINISTRA	funcionário + disciplina	Atribuição docente.

Especialização e sobreposição

Uma pessoa pode ser estudante, funcionária ou ambas. Mariana, código 5, aparece nas duas subclasses. O DDL garante a existência da pessoa referenciada, mas não impõe a especialização total descrita nos comentários: é possível inserir PESSOA sem uma linha em qualquer subclasse.

A orientação é uma FK entre ESTUDANTE e FUNCIONARIO. Conceitualmente conecta dois papéis de pessoa; fisicamente não é uma FK de uma tabela para ela mesma.

Relações muitos para muitos

Exemplo didático • adaptação

```
ESTUDANTE  ← MATRICULA_DISCIPLINA → DISCIPLINA
FUNCIONARIO ← MINISTRA             → DISCIPLINA
```

A tabela de matrícula guarda atributos do vínculo, como semestre e nota. A chave composta permite que o mesmo estudante curse a mesma disciplina em períodos diferentes. MINISTRA não tem semestre ou ano: o modelo enviado não diferencia atribuições docentes por período.

Restrições importantes

Restrição	Exemplo
PRIMARY KEY	Identifica a linha; pode envolver várias colunas.
UNIQUE	documento, matrícula e registro funcional não se repetem.
NOT NULL	Exige valor não nulo em uma coluna.
CHECK	Semestre em 1 ou 2; ano de 1900 a 9999; nota de 0 a 10.
FOREIGN KEY	Exige referência válida, quando a fiscalização está ativa.

Em SQLite, declarar uma coluna como INTEGER não equivale a todas as validações de tipo que você poderia desejar em uma aplicação. Observe a tipagem dinâmica e escreva verificações adicionais quando a regra exigir um domínio mais restrito.

18. SQL de leitura: do básico aos relatórios

SELECT, WHERE e ORDER BY

Exemplo didático • adaptação

```
SELECT codigo, nome, carga_horaria
FROM DISCIPLINA
WHERE carga_horaria > 60
ORDER BY carga_horaria DESC, nome;
```

SELECT escolhe colunas; FROM informa a origem; WHERE filtra linhas; ORDER BY define a ordenação. O segundo critério desempata o primeiro. Sem ORDER BY, não presuma uma ordem estável.

JOIN e LEFT JOIN

Exemplo didático • adaptação

```
SELECT pe.nome AS estudante,
       COALESCE(pf.nome, 'Sem orientador') AS orientador
FROM ESTUDANTE e
JOIN PESSOA pe ON pe.codigo = e.codigo
LEFT JOIN FUNCIONARIO f ON f.codigo = e.id_orientador
LEFT JOIN PESSOA pf ON pf.codigo = f.codigo
ORDER BY pe.nome;
```

JOIN exige correspondência. LEFT JOIN preserva os registros da esquerda sem correspondência à direita. Um filtro inadequado sobre a tabela opcional no WHERE pode excluir essas linhas e eliminar o efeito desejado do LEFT JOIN.

NULL e CASE

Exemplo didático • adaptação

```
SELECT id_estudante,
       CASE
         WHEN nota_final IS NULL THEN 'Cursando'
         WHEN nota_final >= 6 THEN 'Aprovado'
         ELSE 'Reprovado'
       END AS situacao
FROM MATRICULA_DISCIPLINA;
```

Use IS NULL para ausência e IS NOT NULL para presença. Comparar com = NULL não encontra as linhas esperadas. A nota mínima 6 é uma regra adotada pelas consultas do material.

Agregações e filtros de grupo

Exemplo didático • adaptação

```
SELECT disciplina_codigo,
       COUNT(*) AS matriculas,
       COUNT(nota_final) AS avaliados,
       ROUND(AVG(nota_final), 2) AS media
FROM MATRICULA_DISCIPLINA
GROUP BY disciplina_codigo
HAVING AVG(nota_final) >= 8;
```

COUNT(*) conta linhas; COUNT(coluna) conta valores não nulos; AVG, MIN e MAX desconsideram valores nulos. WHERE filtra antes do agrupamento; HAVING filtra grupos. Em LEFT JOIN, conte uma chave da tabela dependente para que a ausência resulte em zero.

Subconsultas e existência

Exemplo didático • adaptação

```
SELECT d.codigo, d.nome
FROM DISCIPLINA d
WHERE NOT EXISTS (
  SELECT 1 FROM MINISTRA m
  WHERE m.disciplina_codigo = d.codigo
);
```

A subconsulta verifica se existe algum vínculo para a disciplina atual. NOT EXISTS encontra disciplinas sem atribuição docente. Esse padrão evita depender de uma contagem quando o objetivo é apenas verificar existência.

19. Depuração e cuidados com o original

Investigue por etapa

- DOM: o ID existe? O elemento foi encontrado no momento correto?
- Entrada: o valor é string, número ou null? A conversão preserva zero?
- Objeto: as propriedades têm os nomes que db.js espera?
- SQL: os parâmetros estão na ordem correta? As referências existem?
- Retorno: houve erro, zero linhas ou um array com registros?
- Tela: o destino está correto e a renderização foi chamada novamente?

Mensagem	Possível causa
no such table	DDL não executado ou banco reinicializado.
UNIQUE constraint failed	Chave ou campo único repetido.
FOREIGN KEY constraint failed	Pessoa, estudante, funcionário ou disciplina referenciada não existe.
CHECK constraint failed	Valor viola uma condição do DDL.
NOT NULL constraint failed	Campo obrigatório recebeu null.
Cannot read properties of null	Elemento não encontrado pelo JavaScript.

Problemas identificados no código enviado

1. Exclusão pela entidade errada. As listagens de estudante, funcionário e disciplina passam codigo como coluna de ação. No mapa deleteHandlers, codigo chama Pessoa.deletar. O botão não distingue corretamente a entidade e pode atingir o registro de pessoa, com efeitos em cascata.

2. Código textual em onclick. O gerador coloca o valor da chave diretamente no JavaScript inline. CC001 pode virar um identificador em vez de uma string e provocar erro. Prefira listener com função que recebe o valor como dado, sem construir código executável em texto.

3. Cabeçalho duplicado. buildTable cria a variável thead já com a tag thead e depois envolve esse resultado em outro thead. A marcação produzida é inválida e depende de correção do navegador.

4. Conteúdo interpretado como HTML. Valores da base e algumas mensagens entram em innerHTML. Use textContent quando a intenção for apresentar texto.

5. Mensagem de sucesso sem conferir linhas. O formulário de orientação não verifica se o estudante foi localizado pelo UPDATE.

6. Comentários divergentes. A inicialização cria banco vazio; o método de exportação exposto chama-se exportDB. Considere a implementação efetiva ao preparar a aula.

O que esta apostila altera
Os recortes documentam o material recebido. As melhorias são apresentadas como propostas; os arquivos originais não foram modificados.

20. Bases de atividade e resultados conferidos

O DML contém INSERT, UPDATE e DELETE. Isso significa que a base após a execução integral difere do estado imediatamente após a carga das seis primeiras seções. Definir o cenário evita que o aluno considere sua consulta errada apenas porque não encontra notas nulas ou estudantes sem orientador.

Cenário	Como preparar	Características
Base A	Executar DDL.sql e todo o DML.sql.	Estado final após as alterações de exemplo.
Base B	Executar DDL.sql e somente as seções 1 a 6 do DML.sql.	Estado anterior aos UPDATE e DELETE.

Tabela	Base A	Base B
PESSOA	6	6
FUNCIONARIO	3	3
ESTUDANTE	4	4
DISCIPLINA	5	5
MINISTRA	5	5
MATRICULA_DISCIPLINA	7	8

O que muda na Base A

• Rodrigo recebe nota 7.5 em CC001, semestre 2 de 2024.

• Rodrigo passa a ser orientado pelo funcionário 1.

• Beatriz recebe um novo endereço.

• A matrícula de Beatriz em CC001, semestre 1 de 2024, é excluída.

Assim, na Base A não há notas nulas nem estudantes sem orientador. Existem seis matrículas aprovadas e uma reprovada. Na Base B, Rodrigo está sem orientador e sua nota em CC001 é nula.

Conferência das consultas de DQL.sql na Base A

Consulta	Linhas	Consulta	Linhas	Consulta	Linhas
1	6	6	2	11	5
2	4	7	7	12	6
3	3	8	5	13	3
4	1	9	4	14	4
5	5	10	0	15	3

Os scripts e as 15 consultas foram executados em SQLite para verificar contagens e resultados. A validação dos dados não equivale a uma homologação completa da interface no navegador.

21. Laboratório: seguir os dados no código

Antes de escrever as consultas finais, pratique a conexão entre os três arquivos. Registre os IDs, as funções e o SQL envolvidos em cada resposta.

L01. Encontre a origem

No HTML, localize p-nome. No JavaScript, encontre a leitura de .value. Explique por que getElementById sozinho não retorna o texto digitado.

L02. Encontre o destino

Localize table-pessoa no HTML. Mostre a função que substitui seu conteúdo e explique o que acontece com a mensagem inicial.

L03. Faça uma tabela de rastreamento

Para documento, nome, estado e complemento, registre ID HTML, propriedade do objeto, transformação e coluna SQL.

L04. Observe os tipos

Leia d-ch e mostre seu typeof antes e depois da conversão. Explique o efeito do campo vazio.

L05. Acompanhe um cadastro

Insira uma disciplina fictícia. Liste a sequência de funções do submit até a atualização da div.

L06. Provoque uma duplicação

Tente repetir o código da disciplina. Identifique em qual camada o erro é detectado e como chega ao toast.

L07. Teste a referência

Tente cadastrar um estudante sem a pessoa correspondente. Explique a relação entre a FK e o erro exibido.

L08. Diferencie zero e ausência

Crie matrículas distintas com nota vazia e nota zero. Consulte ambas e compare null com 0.

L09. Altere um ID

Em uma cópia, renomeie table-disciplina somente no HTML. Identifique a falha e ajuste o JavaScript correspondente.

L10. Preencha para edição

Implemente carregarPessoaParaEdicao. Descreva por que o próximo submit precisa chamar atualizar.

L11. Apresente texto com segurança

Use uma palavra entre sinais de menor e maior como nome fictício. Compare textContent com innerHTML sem executar scripts.

L12. Explique a persistência

Exporte o banco, recarregue a página e descreva o que permaneceu e o que foi perdido. Não use uma base importante para o teste.

22. Mapa de IDs e vocabulário

Origem no HTML	Captura ou ação	Retorno / efeito
form-pessoa / p-*	DB.Pessoa.inserir	table-pessoa
form-estudante / e-*	DB.Estudante.inserir	table-estudante
form-funcionario / f-*	DB.Funcionario.inserir	table-funcionario
form-disciplina / d-*	DB.Disciplina.inserir	table-disciplina
form-matricula / m-*	DB.MatriculaDisciplina.inserir	table-matricula
form-ministra / mn-*	DB.Ministra.inserir	table-ministra
form-orientacao / o-*	DB.Orientacao.definir	table-orientacao
sql-input	DB.query ou DB.run	sql-result-box
script-file-picker	FileReader → executeScript	sql-input e sql-result-box
btn-export-db	DB.exportDB	Download de universidade.db
btn-init-db / btn-clear-db	DB.init	Nova instância vazia

O asterisco representa o grupo de IDs do formulário; não é um seletor executado pelo projeto.

Vocabulário essencial

Termo	Significado neste material
DOM	Objetos que representam o documento HTML.
Evento / listener	Acontecimento e função registrada para responder a ele.
Objeto / array	Conjunto de propriedades / lista de valores.
Callback	Função entregue para execução em resposta a uma etapa ou evento.
Renderizar	Construir ou atualizar a apresentação dos dados.
Parâmetro SQL	Valor associado a um marcador sem concatená-lo ao comando.
PK / FK	Chave primária / chave estrangeira.
Transação	Conjunto de operações tratado como uma unidade.
WASM	Formato usado para executar o motor SQLite no navegador.
API / JSON	Interface de comunicação / formato comum de troca de dados.

23. Exercícios SQL: orientações de entrega

A bateria final tem 50 exercícios. Escreva suas próprias queries antes de abrir DQL.sql: várias soluções de referência já estão nesse arquivo. O objetivo é construir a consulta, justificar os relacionamentos e conferir o resultado.

• Use Base A quando o enunciado não indicar outra base.

• Use Base B nas atividades com notas ainda não lançadas e estudante sem orientador.

• Faça alterações em uma cópia ou restaure a base entre exercícios independentes.

• Os exercícios 36 a 41 formam uma sequência. O exercício 43 remove a matrícula criada nessa sequência.

• O exercício 50 usa um novo código disponível e deve ser executado em uma base de teste.

Formato sugerido

Exemplo didático • adaptação

```
-- Exercício 07: pessoas em ordem alfabética
-- Objetivo: ...
-- Base utilizada: A

-- Escreva sua consulta aqui.

-- Conferência: quantidade de linhas e interpretação.
-- Nos comandos de alteração, consulte antes e depois.
```

Critérios de avaliação

Critério	O que verificar
Correção	A consulta responde ao enunciado e usa o cenário certo.
Relacionamentos	As junções usam as chaves corretas, sem multiplicação acidental.
Integridade	UPDATE e DELETE têm filtros precisos; FKs são respeitadas.
NULL e agregação	Ausência, zero, contagens e médias são tratados corretamente.
Clareza	Aliases, ordenação e formatação tornam a solução compreensível.
Verificação	Há consulta de conferência e explicação dos resultados.

Sem gabarito completo nesta bateria
Os enunciados foram separados das soluções para uso com os alunos. As consultas originais de DQL.sql podem ser usadas pelo professor como referência para parte das atividades. As contagens de conferência aparecem no último bloco.

Exercícios • A — Estrutura e integridade

01. Reconstrução de DISCIPLINA

Em um banco separado, escreva o CREATE TABLE de DISCIPLINA com chave primária textual, nome obrigatório e carga horária obrigatória e positiva.

02. Reconstrução de MINISTRA

Crie MINISTRA com chave primária composta, as duas chaves estrangeiras e as regras de exclusão previstas no DDL.

03. Reconstrução de MATRICULA_DISCIPLINA

Crie a tabela com chave de quatro colunas, FKs e restrições de semestre, ano e nota. A nota deve aceitar NULL.

04. Fiscalização de referências

Escreva o comando SQLite que ativa as chaves estrangeiras na conexão e um comando que consulte esse estado.

05. Violação de nota

Escreva um INSERT com estudante e disciplina existentes, combinação de chave ainda não usada e nota 11. Explique a restrição violada.

06. Violação de unicidade

Tente inserir uma nova pessoa com documento já existente e código ainda não usado, preenchendo corretamente os demais campos. Explique o resultado.

Exercícios • B — Consultas básicas

07. Pessoas em ordem alfabética

Liste código, nome e documento de todas as pessoas, ordenando pelo nome.

08. Pessoas por cidade

Liste nome, cidade e estado das pessoas de Chapecó. Ordene pelo nome.

09. Disciplinas mais extensas

Liste código, nome e carga horária das disciplinas com mais de 60 horas, da maior para a menor carga. Desempate pelo nome.

10. Busca parcial

Liste código e nome das pessoas cujo nome contenha Silva.

11. Busca por curso

Liste código, matrícula e curso dos estudantes de Ciência da Computação.

12. Admissões a partir de 2020

Liste código, registro funcional e data de admissão de funcionários admitidos a partir de 2020-01-01. Ordene pela data.

13. Complemento ausente

Liste nome, logradouro e número das pessoas com complemento NULL.

14. Endereço formatado

Produza nome e uma coluna endereco_completo com logradouro, número, complemento, bairro, cidade, estado e CEP. Evite que complemento NULL anule o endereço.

Exercícios • C — Relacionamentos

15. Estudantes e dados pessoais

Liste nome, documento, matrícula e curso de cada estudante, em ordem alfabética.

16. Funcionários e dados pessoais

Liste nome, registro funcional, setor e data de admissão de cada funcionário.

17. Acúmulo de papéis

Liste pessoas que são estudantes e funcionárias, mostrando código, nome, matrícula e registro funcional.

18. Matrículas em Banco de Dados I

Liste estudante, disciplina, semestre, ano e nota das matrículas em CC001. Ordene por ano e semestre decrescentes e nome crescente.

19. Histórico completo

Liste estudante, código e nome da disciplina, semestre, ano e nota. Ordene por estudante e período crescente.

20. Situação acadêmica

Acrescente ao histórico Cursando, Aprovado ou Reprovado. Considere nota mínima 6 e trate primeiro as notas nulas.

21. Disciplinas ministradas

Liste funcionário, registro funcional, código e nome de cada disciplina que ministra.

22. Orientações existentes

Liste estudante, matrícula e orientador, incluindo somente estudantes que possuem orientador.

23. Todos os orientandos

Liste todos os estudantes e seus orientadores. Para ausência de orientação, apresente Sem orientador.

24. Sem orientador — Base B

Liste código, nome, matrícula e curso dos estudantes sem orientador. Repita na Base A e explique a diferença.

25. Sem docente

Liste código e nome das disciplinas que não aparecem em MINISTRA.

Exercícios • D — Agregações e subconsultas

26. Estudantes por curso

Conte estudantes por curso. Ordene pela quantidade decrescente e pelo curso para desempatar.

27. Médias por disciplina

Mostre código, nome, quantidade de notas lançadas, média com duas casas decimais, menor nota e maior nota por disciplina com avaliação.

28. Média elevada

Liste somente disciplinas com média maior ou igual a 8. Use agrupamento e filtro de grupos.

29. Orientandos por funcionário

Liste todos os funcionários e a quantidade de estudantes que orientam, incluindo quantidade zero.

30. Carga horária docente

Liste funcionário, quantidade de disciplinas ministradas e soma da carga horária. Inclua funcionários sem disciplina com quantidade e carga iguais a zero.

31. Mais de um docente

Liste disciplinas ministradas por mais de um funcionário e a quantidade de docentes de cada uma.

32. Matrículas versus avaliações — Base B

Para cada disciplina, mostre total de matrículas e total de notas lançadas. Inclua disciplinas sem matrícula. Explique COUNT(*) e COUNT(nota_final), e escolha uma contagem adequada ao LEFT JOIN.

33. Acima da média geral

Liste estudantes cuja média individual é maior que a média de todas as notas não nulas do banco. Mostre as duas médias para conferência.

34. Disciplinas com reprovação

Liste, sem repetições, disciplinas com pelo menos uma nota inferior a 6. Use EXISTS.

35. Maior nota por disciplina

Mostre disciplina, estudante e nota dos estudantes com maior nota em cada disciplina. Preserve empates e desconsidere notas nulas.

Exercícios • E — Inserções, alterações e exclusões

36. Nova pessoa

Cadastre uma pessoa fictícia com código 7, documento exclusivo e todos os campos obrigatórios. Consulte o registro criado.

37. Novo estudante

Usando a pessoa 7, cadastre MAT-202407, curso Sistemas de Informação e orientador 1.

38. Nova disciplina

Cadastre WEB01, chamada Desenvolvimento Web, com 80 horas.

39. Nova matrícula

Matricule o estudante 7 em WEB01, semestre 1 de 2025, sem nota final. Considere concluídos os exercícios 36 a 38.

40. Lançamento de nota

Atualize a nota da matrícula do exercício 39 para 8.7, identificando a linha pela chave completa.

41. Troca de orientador

Altere o orientador do estudante 7 para o funcionário 2. Consulte o nome do novo orientador.

42. Endereço de Beatriz

Altere somente logradouro, número e complemento da pessoa 4 para valores fictícios escolhidos. Preserve os demais campos.

43. Cancelamento de matrícula

Exclua somente a matrícula criada no exercício 39. Preserve estudante, pessoa e disciplina. Consulte antes e depois.

44. Remoção de orientação

Deixe o estudante 6 sem orientador, preservando seus cadastros e suas matrículas.

45. Cascata com reversão

Em Base A restaurada, inicie uma transação e exclua PESSOA de código 5. Consulte os efeitos em FUNCIONARIO, ESTUDANTE, MINISTRA e MATRICULA_DISCIPLINA. Execute ROLLBACK e confirme a restauração.

Exercícios • F — Desafios integradores

46. Relatório por estudante

Mostre nome, matrícula, curso, orientador, quantidade de matrículas e média. Inclua estudantes sem matrícula; evite contagens incorretas no LEFT JOIN. Uma média sem notas pode permanecer NULL.

47. Disciplina sem matrícula

Cadastre uma disciplina fictícia sem matrícula. Escreva uma consulta que liste todas as disciplinas e suas quantidades de matrículas, mostrando zero para a recém-criada.

48. Busca para a interface

Escreva uma consulta parametrizada para buscar pessoas por parte do nome. Descreva como ler um input e fornecer o parâmetro do LIKE sem concatenar a entrada à estrutura SQL.

49. Consulta para edição

Escreva uma consulta parametrizada que recupere uma matrícula exata pelos quatro componentes da chave. Relacione as colunas aos IDs HTML que receberiam seus valores.

50. Cadastro com transação

Usando um código novo disponível, escreva BEGIN TRANSACTION, INSERT em PESSOA e INSERT em ESTUDANTE. Mostre o caminho de COMMIT e explique como executar ROLLBACK se a segunda operação falhar.

Conferência parcial • Base A original

Exercício	Resultado de referência
9	Duas disciplinas com mais de 60 horas.
17	Mariana Silva Torres.
18	Duas matrículas em CC001.
22	Quatro estudantes com orientador.
25	ES001.
28	Quatro disciplinas com média ≥ 8.
29	Carlos: 3; Ana Maria: 1; Mariana: 0.
31	CC001, com dois docentes.
34	MAT01.

Na Base B, o exercício 24 encontra Rodrigo. Em CC001, o exercício 32 encontra três matrículas e duas notas lançadas. Restaure a base antes de comparar com essas referências.